Bài toán P-Center

Mô tả, Mô hình ILP, Mô hình SAT và Dữ liệu thực nghiệm

Contents

1 Mô tả bài toán **3**
- 1.1 Bối cảnh và ứng dụng . . . 3
- 1.2 Định nghĩa hình thức . . . 3
- 1.3 Độ phức tạp . . . 3

2 Ví dụ minh họa **3**

3 Các ràng buộc chính **4**

4 Mô hình ILP **4**
- 4.1 Tham số đầu vào . . . 4
- 4.2 Biến quyết định . . . 4
- 4.3 Mô hình . . . 5
- 4.4 Giải thích . . . 5

5 Mô hình SAT **5**
- 5.1 Ý tưởng: Quy về bài toán Set Covering . . . 5
- 5.2 Mã hóa CNF . . . 6
  - 5.2.1 Biến Boolean . . . 6
  - 5.2.2 Mệnh đề phủ (Coverage clauses) . . . 6
  - 5.2.3 Ràng buộc lực lượng: At-Most-p . . . 6
- 5.3 Thuật toán tổng thể . . . 6

6 Các phương pháp tối ưu hóa (Optimisation Approaches) **7**
- 6.1 Non-Incremental SAT (Giải SAT nhiều lần) . . . 7
  - 6.1.1 Ý tưởng . . . 7
  - 6.1.2 Quy trình chi tiết . . . 7
  - 6.1.3 Pseudocode . . . 8
  - 6.1.4 Phân tích . . . 8
- 6.2 Incremental SAT (SAT gia tăng) . . . 8
  - 6.2.1 Ý tưởng . . . 8
  - 6.2.2 Cơ chế Assumption Literals . . . 8
  - 6.2.3 Cấu trúc CNF . . . 9
  - 6.2.4 Pseudocode . . . 9
  - 6.2.5 Phân tích . . . 9
- 6.3 MaxSAT (Tối ưu hóa trực tiếp) . . . 10
  - 6.3.1 Ý tưởng . . . 10
  - 6.3.2 Mã hóa bài toán P-Center . . . 10
  - 6.3.3 Pseudocode . . . 11
  - 6.3.4 Phân tích . . . 11

6.4 So sánh các phương pháp 11

7 Dữ liệu thực nghiệm (Benchmark Datasets) 12

7.1 Bộ pmed (OR-Library) 12

7.2 Bộ TSPLIB 12

7.3 Bộ dữ liệu bổ sung 13

1 Mô tả bài toán

1.1 Bối cảnh và ứng dụng

Bài toán *P-Center* (hay *Vertex P-Center*) là một bài toán tối ưu tổ hợp kinh điển trong lĩnh vực **định vị cơ sở** (*facility location*). Bài toán được ứng dụng rộng rãi trong thực tế:

- **Dịch vụ khẩn cấp**: Đặt trạm cứu hỏa / cứu thương sao cho thời gian phản hồi tối đa là nhỏ nhất.
- **Viễn thông**: Đặt các trạm phát / server sao cho khoảng cách (hoặc độ trễ) lớn nhất tới người dùng được tối thiểu hóa.
- **Y tế**: Đặt các bệnh viện hoặc trung tâm tiêm chủng phục vụ một vùng dân cư.
- **Logistics**: Đặt kho hàng để giảm thiểu khoảng cách vận chuyển xa nhất.

1.2 Định nghĩa hình thức

Định nghĩa 1 (Vertex P-Center Problem)**.** *Cho đồ thị vô hướng có trọng số $G = (V, E, w)$ với $|V| = n$ đỉnh, trong đó $w : E \to \mathbb{R}_{\geq 0}$ là hàm trọng số cạnh. Gọi $d(i, j)$ là khoảng cách đường đi ngắn nhất giữa hai đỉnh i và j. Cho số nguyên dương p ($1 \leq p \leq n$).*

*Bài toán **Vertex P-Center** yêu cầu tìm tập $S \subseteq V$ với $|S| = p$ (gọi là các **center**) sao cho:*

$$\min_{S \subseteq V,\, |S|=p} \; \max_{i \in V} \; \min_{j \in S} \; d(i, j)$$

*Nghĩa là, ta cần chọn p đỉnh làm center để **tối thiểu hóa khoảng cách lớn nhất** từ bất kỳ đỉnh nào đến center gần nhất.*

Giá trị tối ưu $r^* = \max_{i \in V} \min_{j \in S^*} d(i, j)$ được gọi là **bán kính phủ** (*covering radius*) hay **p-radius**.

1.3 Độ phức tạp

Bài toán P-Center là **NP-hard** với $p \geq 2$ trên đồ thị tổng quát [1]. Ngay cả phiên bản quyết định – "*Có tồn tại tập S với $|S| \leq p$ sao cho bán kính phủ $\leq R$?*" – cũng là **NP-complete**. Vì vậy, các phương pháp giải chính xác thường dựa trên ILP, SAT/MaxSAT, hoặc branch-and-bound.

2 Ví dụ minh họa

Ví dụ 1. *Xét đồ thị G gồm 6 đỉnh $V = \{1, 2, 3, 4, 5, 6\}$ với ma trận khoảng cách đường đi ngắn nhất cho trong Table 1. Số center cần chọn: $p = 2$.*

Table 1: Ma trận khoảng cách $d(i, j)$ giữa các đỉnh.

d	1	2	3	4	5	6
1	0	2	5	7	8	9
2	2	0	3	5	6	7
3	5	3	0	2	3	4
4	7	5	2	0	1	2
5	8	6	3	1	0	1
6	9	7	4	2	1	0

Phân tích.

- Nếu chọn $S = \{2, 5\}$:
  - Đỉnh 1: $\min(d(1,2), d(1,5)) = \min(2, 8) = 2$
  - Đỉnh 2: $\min(d(2,2), d(2,5)) = \min(0, 6) = 0$
  - Đỉnh 3: $\min(d(3,2), d(3,5)) = \min(3, 3) = 3$
  - Đỉnh 4: $\min(d(4,2), d(4,5)) = \min(5, 1) = 1$
  - Đỉnh 5: $\min(d(5,2), d(5,5)) = \min(6, 0) = 0$
  - Đỉnh 6: $\min(d(6,2), d(6,5)) = \min(7, 1) = 1$

  Bán kính phủ: $\max(2, 0, 3, 1, 0, 1) = 3$.
- Nếu chọn $S = \{1, 6\}$:
  - Khoảng cách lớn nhất: $\max(\min(0, 9), \min(2, 7), \min(5, 4), \min(7, 2), \min(8, 1), \min(9, 0)) = \max(0, 2, 4, 2, 1, 0) = 4$.

  Bán kính phủ: $4 > 3$, không tốt bằng phương án trên.
- Nghiệm tối ưu: $S^* = \{2, 5\}$ với $r^* = 3$.

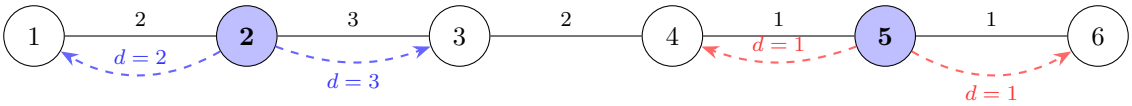

Figure 1: Nghiệm $S^* = \{2, 5\}$ với bán kính phủ $r^* = 3$. Các đỉnh center tô đậm; mũi tên nét đứt thể hiện phân bổ các đỉnh về center gần nhất.

3 Các ràng buộc chính

Bài toán P-Center bao gồm **bốn nhóm ràng buộc** cốt lõi:

(C1) Ràng buộc số lượng center (Cardinality constraint): Phải chọn đúng p đỉnh làm center.

(C2) Ràng buộc gán (Assignment constraint): Mỗi đỉnh phải được gán cho đúng một center.

(C3) Ràng buộc liên kết (Linking constraint): Một đỉnh chỉ có thể được gán cho một center nếu center đó đã được mở.

(C4) Ràng buộc khoảng cách (Distance / minimax constraint): Khoảng cách từ mỗi đỉnh đến center được gán không vượt quá giá trị z (bán kính phủ), đồng thời z được tối thiểu hóa.

4 Mô hình ILP

4.1 Tham số đầu vào

- $V = \{1, 2, \ldots, n\}$: tập đỉnh (vừa là điểm yêu cầu, vừa là vị trí tiềm năng đặt center).
- $d_{ij} \in \mathbb{R}_{\geq 0}$: khoảng cách đường đi ngắn nhất giữa đỉnh i và đỉnh j.
- $p \in \mathbb{Z}_{>0}$: số center cần chọn.

4.2 Biến quyết định

- $y_j \in \{0, 1\}$, $\forall j \in V$: bằng 1 nếu đỉnh j được chọn làm center, 0 ngược lại.
- $x_{ij} \in \{0, 1\}$, $\forall i, j \in V$: bằng 1 nếu đỉnh i được gán cho center j, 0 ngược lại.
- $z \in \mathbb{R}_{\geq 0}$: bán kính phủ (khoảng cách lớn nhất).

4.3 Mô hình

$$
\begin{aligned}
\min \quad & z && && (1)\\
\text{s.t.} \quad & \sum_{j \in V} y_j = p && && (2)\\
& \sum_{j \in V} x_{ij} = 1 && \forall\, i \in V && (3)\\
& x_{ij} \le y_j && \forall\, i, j \in V && (4)\\
& \sum_{j \in V} d_{ij}\, x_{ij} \le z && \forall\, i \in V && (5)\\
& y_j \in \{0, 1\} && \forall\, j \in V && (6)\\
& x_{ij} \in \{0, 1\} && \forall\, i, j \in V && (7)\\
& z \ge 0 && && (8)
\end{aligned}
$$

4.4 Giải thích

- (1): Hàm mục tiêu – tối thiểu hóa bán kính phủ z.
- (2): **(C1)** – Chọn đúng p center.
- (3): **(C2)** – Mỗi đỉnh i được gán cho đúng một center.
- (4): **(C3)** – Đỉnh i chỉ được gán cho j nếu j là center ($y_j = 1$).
- (5): **(C4)** – Khoảng cách từ đỉnh i đến center được gán không vượt quá z.
- (6)–(8): Miền giá trị biến.

Nhận xét. Mô hình có $O(n^2)$ biến nhị phân và $O(n^2)$ ràng buộc. Trong thực tế, có thể cải thiện bằng cách:

- Loại bỏ các biến x_{ij} khi $d_{ij} > z_{\text{UB}}$ (upper bound biết trước).
- Sử dụng các bất đẳng thức hợp lệ (*valid inequalities*) và kỹ thuật *branch-and-cut*.

5 Mô hình SAT

Thay vì giải bài toán tối ưu trực tiếp, cách tiếp cận SAT biến đổi P-Center thành **một chuỗi bài toán quyết định** thông qua tìm kiếm nhị phân trên bán kính R.

5.1 Ý tưởng: Quy về bài toán Set Covering

Bước 1: Sắp xếp các giá trị khoảng cách. Gọi $D = \{d_{ij} : i, j \in V, i \neq j\}$ là tập tất cả giá trị khoảng cách phân biệt, sắp xếp tăng dần: $d^{(1)} < d^{(2)} < \cdots < d^{(L)}$.

Bước 2: Tìm kiếm nhị phân. Sử dụng **tìm kiếm nhị phân** trên tập D để tìm giá trị R^* nhỏ nhất sao cho bài toán quyết định sau có lời giải:

> *"Có tồn tại tập $S \subseteq V$ với $|S| \le p$ sao cho mọi đỉnh $i \in V$ đều có ít nhất một center $j \in S$ với $d(i, j) \le R$?"*

Bài toán quyết định này chính là bài toán **Set Covering**: với mỗi đỉnh j, định nghĩa *tập phủ*:

$$C_j(R) = \{i \in V : d(i, j) \le R\}$$

Cần tìm $\le p$ tập C_j sao cho hợp của chúng phủ toàn bộ V.

Bước 3: Mã hóa SAT. Bài toán Set Covering được mã hóa thành công thức CNF và giải bằng SAT solver.

5.2 Mã hóa CNF

5.2.1 Biến Boolean

- $y_j \in \{\texttt{true}, \texttt{false}\}$, $\forall j \in V$: $y_j = \texttt{true}$ nếu đỉnh j được chọn làm center.

5.2.2 Mệnh đề phủ (Coverage clauses)

Với mỗi đỉnh $i \in V$, phải tồn tại ít nhất một center trong bán kính R:

$$\bigvee_{j \in V:\, d(i,j) \le R} y_j \qquad \forall\, i \in V \tag{9}$$

Mỗi ràng buộc (9) tương ứng với **một mệnh đề** (clause) trong CNF. Tổng cộng có n mệnh đề phủ.

5.2.3 Ràng buộc lực lượng: At-Most-p

Cần đảm bảo $\sum_{j \in V} y_j \le p$, tức tối đa p biến y_j nhận giá trị `true`.

Ràng buộc này được mã hóa bằng **Sequential Counter** (Sinz, 2005 [4]):

Biến phụ. Đặt $s_{j,k}$ ($j = 1, \ldots, n$; $k = 1, \ldots, p$) là biến phụ, trong đó $s_{j,k} = \texttt{true}$ nghĩa là "trong số $y_1, \ldots, y_j$ có ít nhất k biến bằng `true`".

Mệnh đề mã hóa.

$$\neg y_1 \vee s_{1,1} \tag{10}$$
$$\neg y_j \vee s_{j,1} \qquad \forall\, j = 2, \ldots, n \tag{11}$$
$$\neg s_{j-1,1} \vee s_{j,1} \qquad \forall\, j = 2, \ldots, n \tag{12}$$
$$\neg y_j \vee \neg s_{j-1,k-1} \vee s_{j,k} \qquad \forall\, j = 2, \ldots, n;\ k = 2, \ldots, p \tag{13}$$
$$\neg s_{j-1,k} \vee s_{j,k} \qquad \forall\, j = 2, \ldots, n;\ k = 2, \ldots, p \tag{14}$$
$$\neg y_j \vee \neg s_{j-1,p} \qquad \forall\, j = 2, \ldots, n \tag{15}$$

- (10)–(12): Khởi tạo và truyền bit đầu tiên.
- (13)–(14): Truyền đếm: nếu y_j bằng true và đã có $k-1$ biến true, thì $s_{j,k}$ phải true.
- (15): **Mệnh đề cấm**: nếu đã có p biến true ($s_{j-1,p}$), thì y_j phải false $\Rightarrow$ đảm bảo at-most-p.

Mã hóa Sequential Counter thêm $O(np)$ biến phụ và $O(np)$ mệnh đề.

5.3 Thuật toán tổng thể

Phân tích.

- Số lần gọi SAT solver: $O(\log L)$ với $L = |D| \le \binom{n}{2}$.
- Mỗi lần gọi, CNF có n mệnh đề phủ + $O(np)$ mệnh đề cardinality.
- Phương pháp này rất hiệu quả trong thực tế nhờ sức mạnh của các SAT solver hiện đại (CaDiCaL, Glucose, MiniSat, ...).

Algorithm 1 Giải P-Center bằng SAT

Require: Đồ thị $G = (V, E, w)$, số center p
Ensure: Tập center tối ưu S^*, bán kính tối ưu r^*

```
Tính ma trận khoảng cách ngắn nhất d_ij, ∀i, j ∈ V
D ← tập giá trị d_ij phân biệt, sắp xếp tăng dần
lo ← 1, hi ← |D|
while lo < hi do
    mid ← ⌊(lo + hi)/2⌋
    R ← D[mid]
    Xây dựng CNF: mệnh đề phủ (9) + at-most-p (10)–(15)
    if SAT_Solver(CNF) trả về SAT then
        hi ← mid
        Lưu nghiệm S
    else
        lo ← mid + 1
    end if
end while
return S*, r* = D[lo]
```

6 Các phương pháp tối ưu hóa (Optimisation Approaches)

Bài toán P-Center là bài toán *tối ưu* (minimisation), nhưng SAT solver chỉ giải bài toán *quyết định* (decision). Do đó, cần một chiến lược bọc ngoài (*outer loop*) để chuyển bài toán tối ưu thành chuỗi bài toán quyết định.

Phần này trình bày ba phương pháp chính: **Non-Incremental SAT**, **Incremental SAT**, và **MaxSAT**.

6.1 Non-Incremental SAT (Giải SAT nhiều lần)

6.1.1 Ý tưởng

Đây là phương pháp đơn giản nhất: sử dụng **tìm kiếm nhị phân** trên tập giá trị bán kính $D = \{d^{(1)}, d^{(2)}, \ldots, d^{(L)}\}$. Tại mỗi bước, **xây dựng lại toàn bộ công thức CNF từ đầu** và gọi một SAT solver mới.

6.1.2 Quy trình chi tiết

1. **Tiền xử lý**: Tính ma trận d_{ij}, thu thập tập D các giá trị khoảng cách phân biệt, sắp xếp tăng dần.
2. **Tìm kiếm nhị phân**: Đặt $lo = 1$, $hi = |D|$.
3. Tại mỗi bước $mid = \lfloor (lo + hi)/2 \rfloor$, đặt $R = D[mid]$:
   (a) Xây dựng **CNF mới hoàn toàn**:
      - n mệnh đề phủ: $\bigvee_{j:d(i,j) \le R} y_j$ cho mỗi i.
      - Mã hóa at-most-p (ví dụ Sequential Counter): $O(np)$ mệnh đề.

   (b) Khởi tạo một **SAT solver mới** và nạp CNF.
   (c) Gọi solver:
      - Nếu **SAT**: $hi \leftarrow mid$, lưu nghiệm.
      - Nếu **UNSAT**: $lo \leftarrow mid + 1$.

   (d) **Hủy solver** (toàn bộ trạng thái nội bộ, learned clauses bị mất).
4. Lặp lại cho đến $lo = hi$. Trả về $R^* = D[lo]$.

6.1.3 Pseudocode

Algorithm 2 Non-Incremental SAT cho P-Center

Require: Đồ thị $G = (V, E, w)$, số center p
Ensure: Tập center S^*, bán kính tối ưu r^*

```
Tính d_ij, ∀i, j ∈ V
D ← sorted_unique({d_ij})
lo ← 1, hi ← |D|, S* ← ∅
while lo < hi do
    mid ← ⌊(lo + hi)/2⌋
    R ← D[mid]
    φ ← ∅                                          ▷ Xây CNF mới từ đầu
    for all i ∈ V do
        φ ← φ ∪ { ⋁_{j:d(i,j)≤R} y_j }             ▷ Coverage clause
    end for
    φ ← φ ∪ SeqCounter(y_1, ..., y_n, p)           ▷ At-most-p
    solver ← new SAT_Solver()
    solver.add_clauses(φ)
    if solver.solve() = SAT then
        hi ← mid
        S* ← {j : solver.value(y_j) = true}
    else
        lo ← mid + 1
    end if
    delete solver                                  ▷ Hủy solver, mất toàn bộ learned clauses
end while
return S*, r* = D[lo]
```

6.1.4 Phân tích

- **Số lần gọi solver**: $O(\log L)$, với $L \leq \binom{n}{2}$.
- **Chi phí mỗi lần**: Xây dựng CNF $O(n + np)$ + giải SAT (exponential worst case, nhưng thường nhanh trong thực tế).
- **Nhược điểm chính**: Mỗi lần gọi solver đều bắt đầu từ đầu – **mất toàn bộ learned clauses** từ lần giải trước. Đây là lãng phí lớn vì các CNF giữa hai bước liên tiếp thường rất giống nhau (chỉ khác ở các mệnh đề phủ).
- **Ưu điểm**: Đơn giản, dễ cài đặt, không phụ thuộc API đặc thù của solver.

6.2 Incremental SAT (SAT gia tăng)

6.2.1 Ý tưởng

Quan sát then chốt: trong quá trình tìm kiếm nhị phân, **ràng buộc cardinality (at-most-p) không thay đổi** – chỉ có các **mệnh đề phủ thay đổi** khi R thay đổi. Incremental SAT **giữ nguyên solver** xuyên suốt quá trình, tận dụng:

- **Learned clauses**: Các mệnh đề học được từ lần giải trước vẫn hợp lệ (vì chúng là hệ quả logic của các mệnh đề cố định).
- **Heuristic state**: Thứ tự biến (VSIDS scores), phase saving, ...

6.2.2 Cơ chế Assumption Literals

SAT solver hiện đại hỗ trợ cơ chế **assumption** (giả thiết) qua giao diện IPASIR [9]:

- `solver.add_clause(clause)`: Thêm mệnh đề **vĩnh viễn** (không thể gỡ bỏ).
- `solver.solve(assumptions=[a1, a2, ...])`: Giải dưới giả thiết các literal $a_1, a_2, \ldots$ phải đúng. Khi lần gọi kết thúc, các assumption được **tự động gỡ bỏ**.

Kỹ thuật. Với mỗi giá trị bán kính R_t (ứng với bước t trong tìm kiếm nhị phân), tạo một **assumption literal** α_t. Các mệnh đề phủ tại bước t được "bảo vệ" bởi α_t:

$$\neg\alpha_t \vee \bigvee_{j:d(i,j)\leq R_t} y_j \qquad \forall\, i \in V \tag{16}$$

Khi gọi solver với assumption $\alpha_t = \texttt{true}$, các mệnh đề này được kích hoạt. Ở bước tiếp theo, α_t không được giả thiết nữa nên các mệnh đề cũ trở thành **tautology** (luôn đúng) và không ảnh hưởng đến solver.

6.2.3 Cấu trúc CNF

- **Mệnh đề cố định** (thêm một lần duy nhất):
  - At-most-p: Sequential Counter (10)–(15) trên $y_1, \ldots, y_n$.
- **Mệnh đề có điều kiện** (thêm tại mỗi bước t):
  - Coverage guarded: $(\neg\alpha_t \vee y_{j_1} \vee y_{j_2} \vee \cdots)$ cho mỗi i với $\{j_1, j_2, \ldots\} = \{j : d(i,j) \leq R_t\}$.
- **Gọi solver**: `solver.solve(assumptions=[`α_t`])`.

6.2.4 Pseudocode

Algorithm 3 Incremental SAT cho P-Center

Require: Đồ thị $G = (V, E, w)$, số center p
Ensure: Tập center S^*, bán kính tối ưu r^*

```
Tính d_ij, ∀i, j ∈ V
D ← sorted_unique({d_ij})
solver ← new SAT_Solver()                          ▷ Khởi tạo một lần duy nhất
solver.add_clauses(SeqCounter(y_1, ..., y_n, p))   ▷ Mệnh đề cố định
lo ← 1, hi ← |D|, S* ← ∅
while lo < hi do
    mid ← ⌊(lo + hi)/2⌋
    R ← D[mid]
    α ← fresh_variable()                           ▷ Assumption literal mới
    for all i ∈ V do
        solver.add_clause(¬α ∨ ⋁_{j:d(i,j)≤R} y_j)  ▷ Guarded coverage
    end for
    if solver.solve(assumptions=[α]) = SAT then
        hi ← mid
        S* ← {j : solver.value(y_j) = true}
    else
        lo ← mid + 1
    end if                                         ▷ Solver giữ nguyên learned clauses & heuristics
end while
return S*, r* = D[lo]
```

6.2.5 Phân tích

- **Solver chỉ khởi tạo một lần**: Tiết kiệm chi phí khởi tạo.

- **Learned clauses được tích lũy**: Solver ngày càng "thông minh hơn" qua các bước.
- **Mệnh đề cố định chiếm phần lớn CNF**: At-most-p encoding ($O(np)$ mệnh đề) chỉ thêm một lần; mỗi bước chỉ thêm n mệnh đề phủ mới.
- **Chi phí phụ**: Tổng cộng thêm $O(n \log L)$ mệnh đề guarded vào solver (không gỡ bỏ được, nhưng trở thành tautology).
- **Yêu cầu**: SAT solver hỗ trợ API incremental (IPASIR): CaDiCaL, Glucose, MiniSat, Lingeling, ...

Tại sao learned clauses vẫn hợp lệ? Khi giả thiết α_t bị gỡ, mệnh đề $(\neg\alpha_t \vee \ell_1 \vee \cdots \vee \ell_k)$ trở thành tautology (đúng với mọi assignment). Bất kỳ mệnh đề nào suy ra (*resolve*) từ tập mệnh đề gốc + tautology đều vẫn là hệ quả logic hợp lệ. Do đó, **tất cả learned clauses đều an toàn để giữ lại**.

6.3 MaxSAT (Tối ưu hóa trực tiếp)

6.3.1 Ý tưởng

Thay vì giải nhiều bài toán quyết định, MaxSAT cho phép **mã hóa trực tiếp mục tiêu tối ưu** vào một công thức duy nhất. Ta sử dụng **Weighted Partial MaxSAT** (WPMS) [10]:

- **Hard clauses**: Phải thỏa mãn (vi phạm $\Rightarrow$ nghiệm không hợp lệ).
- **Soft clauses**: Có thể vi phạm, mỗi vi phạm chịu **trọng số** (weight). Mục tiêu: tối thiểu hóa tổng trọng số vi phạm.

6.3.2 Mã hóa bài toán P-Center

Biến Boolean.

- y_j, $\forall j \in V$: đỉnh j được chọn làm center.
- t_k, $k = 1, \ldots, L$: biến "ngưỡng" (*threshold*), trong đó $t_k = \texttt{true}$ nghĩa là bán kính tối ưu $\leq d^{(k)}$.

Hard clauses (phải thỏa mãn).

(H1) **At-most-p**: Mã hóa $\sum_j y_j \leq p$ bằng Sequential Counter (như §5).

(H2) **Liên kết ngưỡng – phủ**: Nếu $t_k = \texttt{true}$ (bán kính $\leq d^{(k)}$), thì mọi đỉnh phải được phủ bởi ít nhất một center trong phạm vi $d^{(k)}$:

$$\neg t_k \vee \bigvee_{j:d(i,j)\leq d^{(k)}} y_j \qquad \forall\, i \in V,\ k = 1, \ldots, L \tag{17}$$

(H3) **Đơn điệu ngưỡng**: Nếu bán kính $\leq d^{(k)}$ thì cũng $\leq d^{(k+1)}$:

$$\neg t_k \vee t_{k+1} \qquad \forall\, k = 1, \ldots, L-1 \tag{18}$$

Soft clauses (tối ưu hóa).

Mỗi biến t_k là soft clause đơn vị $\{t_k\}$ với trọng số w_k:

$$\text{soft clause: } (t_k) \quad \text{weight } w_k \qquad \forall\, k = 1, \ldots, L \tag{19}$$

Nếu t_k vi phạm (tức $t_k = \texttt{false}$, nghĩa là bán kính $> d^{(k)}$), ta chịu chi phí w_k.

Chọn trọng số. Hai cách chọn trọng số phổ biến:

- **Trọng số đồng nhất**: $w_k = 1$, $\forall k$. MaxSAT solver sẽ tối đa số t_k đúng, tức **tối thiểu** k^* nhỏ nhất mà $t_{k^*} = \texttt{false}$. Nhờ ràng buộc đơn điệu (18), nghiệm sẽ có dạng $t_1 = \cdots = t_{k^*-1} = \texttt{false}$, $t_{k^*} = \cdots = t_L = \texttt{true}$, tương ứng bán kính tối ưu $r^* = d^{(k^*)}$.
- **Trọng số lũy thừa**: $w_k = 2^{L-k}$. Ưu tiên mạnh việc thỏa mãn các t_k nhỏ (bán kính nhỏ), đảm bảo solver tìm k^* nhỏ nhất trước.

6.3.3 Pseudocode

Algorithm 4 Weighted Partial MaxSAT cho P-Center

Require: Đồ thị $G = (V, E, w)$, số center p
Ensure: Tập center S^*, bán kính tối ưu r^*

```
Tính d_ij, ∀i, j ∈ V
D ← sorted_unique({d_ij}),  L ← |D|
solver ← new MaxSAT_Solver()

                                                        ▷ Hard clauses
solver.add_hard(SeqCounter(y_1, ..., y_n, p))           ▷ At-most-p
for all k = 1, ..., L do
    for all i ∈ V do
        solver.add_hard(¬t_k ∨ ⋁_{j:d(i,j)≤D[k]} y_j)   ▷ Coverage
    end for
end for
for all k = 1, ..., L − 1 do
    solver.add_hard(¬t_k ∨ t_{k+1})                     ▷ Monotonicity
end for

                                                        ▷ Soft clauses
for all k = 1, ..., L do
    solver.add_soft(t_k, w_k)                           ▷ Tối đa hóa số t_k đúng
end for

solver.solve()
S* ← {j : solver.value(y_j) = true}
k* ← min{k : solver.value(t_k) = true}
return S*, r* = D[k*]
```

6.3.4 Phân tích

- **Một lần gọi solver duy nhất**: Không cần outer loop (binary search).
- **Kích thước công thức**: $O(np)$ hard clauses (cardinality) + $O(nL)$ hard clauses (coverage) + L soft clauses + L monotonicity clauses.
- **Ưu điểm**: MaxSAT solver (RC2, Open-WBO, EvalMaxSAT, ...) có các kỹ thuật tối ưu nội bộ mạnh mẽ (stratification, core-guided search, ...) giúp không cần duyệt qua tất cả L giá trị.
- **Nhược điểm**: Kích thước CNF lớn hơn đáng kể ($O(nL)$ vs $O(n)$ cho mỗi bước SAT), có thể chậm hơn trên instance lớn nếu L lớn.

6.4 So sánh các phương pháp

Hướng dẫn chọn phương pháp.

Table 2: So sánh ba phương pháp tối ưu hóa dựa trên SAT cho P-Center.

Tiêu chí	**Non-Incremental**	**Incremental SAT**	**MaxSAT**
Số lần gọi solver	$O(\log L)$ lần, mỗi lần solver mới	$O(\log L)$ lần, cùng solver	1 lần duy nhất
Learned clauses	Mất sau mỗi bước	Tích lũy xuyên suốt	Nội bộ MaxSAT solver
Kích thước CNF mỗi bước	$O(n + np)$	$O(n)$ mới + $O(np)$ cố định	$O(nL + np)$ tổng
Yêu cầu API	API cơ bản	IPASIR (assumptions)	WCNF (hard/soft)
Solver phù hợp	MiniSat, CaDiCaL, Glucose	CaDiCaL, Lingeling, Glucose	RC2, Open-WBO, Eval-MaxSAT
Độ phức tạp cài đặt	Đơn giản	Trung bình	Trung bình
Hiệu quả thực tế	Tốt cho instance nhỏ	Tốt nhất cho instance trung bình – lớn	Tốt khi L không quá lớn

- Nếu cần **cài đặt nhanh, đơn giản**: dùng **Non-Incremental**.
- Nếu muốn **hiệu quả tốt nhất** trên instance lớn: dùng **Incremental SAT** (tận dụng learned clauses).
- Nếu muốn **formulation gọn, một lần gọi**: dùng **MaxSAT** (nhưng cần cân nhắc kích thước L).

7 Dữ liệu thực nghiệm (Benchmark Datasets)

Các nghiên cứu về P-Center thường đánh giá trên hai bộ dữ liệu chuẩn chính:

7.1 Bộ pmed (OR-Library)

- **Nguồn**: OR-Library, xây dựng bởi Beasley [2].
- **Nội dung**: 40 instance (`pmed1` – `pmed40`).
- **Quy mô**: Số đỉnh n từ 100 đến 900; giá trị p từ 5 đến 200.
- **Cấu trúc**: Đồ thị thưa (sparse graph), khoảng cách giữa các cặp đỉnh được tính bằng đường đi ngắn nhất (thuật toán Floyd-Warshall hoặc Dijkstra).
- **Đặc điểm**:
  - Ban đầu thiết kế cho bài toán *p-median*, nhưng được sử dụng phổ biến cho p-center vì cùng cấu trúc đồ thị.
  - Nghiệm tối ưu đã được biết cho tất cả 40 instance.
- **URL**: `https://people.brunel.ac.uk/~mastjjb/jeb/orlib/pmedinfo.html`

7.2 Bộ TSPLIB

- **Nguồn**: TSPLIB, xây dựng bởi Reinelt [3].
- **Nội dung**: Các instance gốc dùng cho bài toán TSP, được chuyển đổi cho p-center bằng cách tính khoảng cách Euclid giữa các điểm.
- **Quy mô**: Từ vài trăm đến hàng nghìn đỉnh (ví dụ: `att48`, `eil76`, `lin318`, `rat783`, `pr1002`, ...).
- **Đặc điểm**:
  - Đồ thị đầy đủ (complete graph) với khoảng cách Euclid 2D.
  - Khó hơn bộ pmed do đồ thị đầy đủ $\Rightarrow$ ma trận khoảng cách dày đặc.

- **URL**: `http://comopt.ifi.uni-heidelberg.de/software/TSPLIB95/`

7.3 Bộ dữ liệu bổ sung

Ngoài hai bộ chuẩn trên, một số nghiên cứu gần đây đã đề xuất các instance lớn hơn. Ví dụ, Liu et al. [5] thử nghiệm trên cả pmed và TSPLIB, sử dụng phương pháp quy về Set Covering + SAT solver để giải hiệu quả các instance cỡ lớn.

Table 3: Tổng hợp bộ dữ liệu benchmark cho P-Center.

Bộ dữ liệu	**Loại đồ thị**	**Số instance**	**Quy mô n**	**Tham khảo**
pmed (OR-Library)	Thưa (sparse)	40	100 – 900	[2]
TSPLIB	Đầy đủ (Euclid 2D)	nhiều	48 – 3038+	[3]

References

[1] O. Kariv and S. L. Hakimi. An algorithmic approach to network location problems. I: The p-centers. *SIAM Journal on Applied Mathematics*, 37(3):513–538, 1979.

[2] J. E. Beasley. OR-Library: distributing test problems by electronic mail. *Journal of the Operational Research Society*, 41(11):1069–1072, 1990.

[3] G. Reinelt. TSPLIB — a traveling salesman problem library. *ORSA Journal on Computing*, 3(4):376–384, 1991.

[4] C. Sinz. Towards an optimal CNF encoding of boolean cardinality constraints. In *Proceedings of the 11th International Conference on Principles and Practice of Constraint Programming (CP 2005)*, pages 827–831, 2005.

[5] X. Liu, Y. Fang, J. Chen, and Z. Lü. Effective approaches to solve P-Center problem via set covering and SAT. *arXiv preprint arXiv:2008.xxxx*, 2020.

[6] E. Minieka. The m-center problem. *SIAM Review*, 12(1):138–139, 1970.

[7] M. S. Daskin. *Network and Discrete Location: Models, Algorithms, and Applications*. John Wiley & Sons, 1995.

[8] H. Çalık, M. Labbé, and H. Yaman. p-center problems. In G. Laporte, S. Nickel, and F. Saldanha da Gama, editors, *Location Science*, chapter 4, pages 79–92. Springer, 2015.

[9] T. Balyo, A. Biere, M. Iser, and C. Sinz. SAT Race 2015. *Artificial Intelligence*, 241:45–65, 2016. (Defines the IPASIR incremental SAT interface.)

[10] C. M. Li and F. Manya. MaxSAT, hard and soft constraints. In A. Biere, M. Heule, H. van Maaren, and T. Walsh, editors, *Handbook of Satisfiability*, chapter 19, pages 903–927. IOS Press, 2nd edition, 2021.

[11] N. Eén and N. Sörensson. An extensible SAT-solver. In *SAT 2003*, LNCS 2919, pages 502–518. Springer, 2003. (MiniSat – foundational incremental SAT solver.)